

CS-114 Fundamentals of Programming

Instructor: Jahan

Zeb

Department of Computer Engineering (DCE)

College of E&ME

NUST

Logical Operators

- Used as conditions in loops, if statements

- **&&** (logical **AND**)

- **true** if both conditions are **true**

- ```
if (student == 1 && marks >= 70)
 ++goodstudents;
```

- **||** (logical **OR**)

- **true** if either of condition is **true**

- ```
if ( semesterAverage >= 90 || finalExam >= 90 )  
    cout << "Student grade is A" << endl;
```

Logical Operators

□ ! (logical **NOT**, logical negation)

- Returns **true** when its condition is **false**, & vice versa

```
if ( !( grade == sentinelValue ) )  
    cout << "The next grade is " << grade << endl;
```

Alternative:

```
if ( grade != sentinelValue )  
    cout << "The next grade is " << grade << endl;
```

Confusing Equality (==) and Assignment (=) Operators

□ Common error

- Does not typically cause syntax errors

□ Aspects of problem

- Expressions that have a value can be used for decision
 - Zero = false, nonzero = true
- Assignment statements produce a value (the value to be assigned)

Confusing Equality (==) and Assignment (=) Operators

□ Example

```
if ( payCode == 4 )  
    cout << "You get a bonus!" << endl;
```

- If paycode is 4, bonus given

□ If == was replaced with =

```
if ( payCode = 4 )  
    cout << "You get a bonus!" << endl;
```

- Paycode set to 4 (no matter what it was before)
- Statement is true (since 4 is non-zero)
- Bonus given in every case

Assignment Operators

□ Assignment expression abbreviations

- Addition assignment operator

`c = c + 3;` abbreviated to
`c += 3;`

□ Statements of the form

`variable = variable operator expression;`

can be rewritten as

`variable operator= expression;`

□ Other assignment operators

`d -= 4` (`d = d - 4`)

`e *= 5` (`e = e * 5`)

`f /= 3` (`f = f / 3`)

`g %= 9` (`g = g % 9`)

Increment and Decrement Operators

- Increment operator (**++**)
 - Increment variable by one
 - **c++**
 - Same as **c += 1**
- Decrement operator (**--**) similar
 - Decrement variable by one
 - **c--**
 - Same as **c -= 1**

Increment and Decrement Operators

□ Preincrement

- Variable changed before used in expression
 - Operator before variable (**++c** or **--c**)

□ Postincrement

- Incremented changed after expression
 - Operator after variable (**c++**, **c--**)

Increment and Decrement Operators

□ If **c = 5**, then

– **cout << ++c;**

- **c** is changed to **6**, then printed out

– **cout << c++;**

- Prints out **5** (**cout** is executed before the increment).
- **c** then becomes **6**

Increment and Decrement Operators

□ When variable not in expression

- Preincrementing and postincrementing have same effect

```
++c;
```

```
cout << c;
```

and

```
c++;
```

```
cout << c;
```

are the same

```
1 // Fig. 2.14: fig02_14.cpp
2 // Preincrementing and postincrementing.
3 #include <iostream>
4
5 using std::cout;
6 using std::endl;
7
8 // function main begins program execution
9 int main()
10 {
11     int c;                // declare variable
12
13     // demonstrate postincrement
14     c = 5;                // assign 5 to c
15     cout << c << endl;    // print 5
16     cout << c++ << endl;   // print 5 then postincrement
17     cout << c << endl << endl; // print 6
18
19     // demonstrate preincrement
20     c = 5;                // assign 5 to c
21     cout << c << endl;    // print 5
22     cout << ++c << endl;   // preincrement then print 6
23     cout << c << endl;    // print 6
```

```
24
25     return 0;    // indicate successful termination
26
27 } // end function main
```

5

5

6

5

6

6

Essentials of Counter-Controlled Repetition

- Counter-controlled repetition requires
 - Name of control variable/loop counter
 - Initial value of control variable
 - Condition to test for final value
 - Increment/decrement to modify control variable when looping

```
1 // Fig. 2.16: fig02_16.cpp
2 // Counter-controlled repetition.
3 #include <iostream>
4
5 using std::cout;
6 using std::endl;
7
8 // function main begins program execution
9 int main()
10 {
11     int counter = 1;           // initialization
12
13     while ( counter <= 10 ) {   // repetition condition
14         cout << counter << endl; // display counter
15         ++counter;              // increment
16
17     } // end while
18
19     return 0; // indicate successful termination
20
21 } // end function main
```

1	
2	
3	
4	
5	
6	
7	
8	
9	
10	

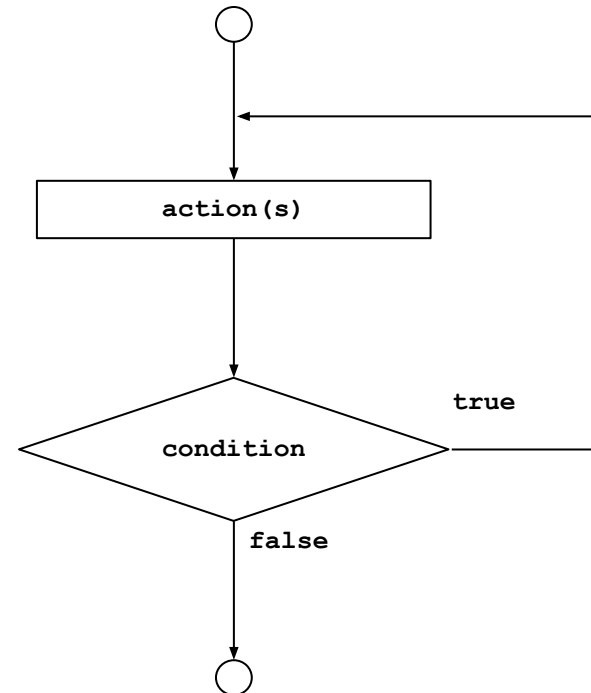
do/while Repetition Structure

□ Similar to **while** structure

- Makes loop continuation test at end, not beginning
- Loop body executes at least once

□ Format

```
do {  
    statement  
} while ( condition );
```




```

1 // Fig. 2.24: fig02_24.cpp
2 // Using the do/while repetition structure.
3 #include <iostream>
4
5 using std::cout;
6 using std::endl;
7
8 // function main begins program execution
9 int main()
10 {
11     int counter = 1;           // initialize counter
12
13     do {
14         cout << counter << " "; // display counter
15     } while ( ++counter <= 10 ); // end do/while
16
17     cout << endl;
18
19     return 0; // indicate successful termination
20
21 } // end function main

```

```

1  2  3  4  5  6  7  8  9  10

```

```
#include <iostream>
using namespace std;

int main()
{
    int score1, score2, score3; // Three scores
    double average;             // Average score
    char again;                 // To hold Y or N input

    do
    {
        // Get three scores.
        cout << "Enter 3 scores and I will average them: ";
        cin >> score1 >> score2 >> score3;

        // Calculate and display the average.
        average = (score1 + score2 + score3) / 3.0;
        cout << "The average is " << average << ".\n";

        // Does the user want to average another set?
        cout << "Do you want to average another set? (Y/N) ";
        cin >> again;
    } while (again == 'Y' || again == 'y');
    return 0;
}
```

Enter 3 scores and I will average them: **80 90 70 [Enter]**

The average is 80.

Do you want to average another set? (Y/N) **y [Enter]**

Enter 3 scores and I will average them: **60 75 88 [Enter]**

The average is 74.3333.

Do you want to average another set? (Y/N) **n [Enter]**

for Repetition Structure

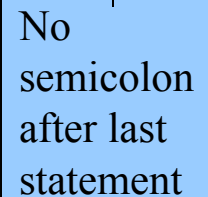
□ General format when using **for** loops

```
for ( initialization; LoopContinuationTest;  
    increment )  
    statement
```

□ Example

```
for( int counter = 1; counter <= 10; counter++ )  
    cout << counter << endl;
```

– Prints integers from one to ten



No
semicolon
after last
statement

```
1 // Fig. 2.17: fig02_17.cpp
2 // Counter-controlled repetition with the for structure.
3 #include <iostream>
4
5 using std::cout;
6 using std::endl;
7
8 // function main begins program execution
9 int main()
10 {
11     // Initialization, repetition condition and incrementing
12     // are all included in the for structure header.
13
14     for ( int counter = 1; counter <= 10; counter++ )
15         cout << counter << endl;
16
17     return 0;    // indicate successful termination
18
19 } // end function main
```



1
2
3
4
5
6
7
8
9
10

for Repetition Structure

- **for** loops can usually be rewritten as **while** loops

```
initialization;  
while ( loopContinuationTest) {  
    statement  
    increment;  
}
```

- Initialization and increment

- For multiple variables, use comma-separated lists

```
for (int i = 0, j = 0; j + i <= 10; j++, i++)  
    cout << j + i << endl;
```

```

1 // Fig. 2.20: fig02_20.cpp
2 // Summation with for.
3 #include <iostream>
4
5 using std::cout;
6 using std::endl;
7
8 // function main begins program execution
9 int main()
10 {
11     int sum = 0;                // initialize sum
12
13     // sum even integers from 2 through 100
14     for ( int number = 2; number <= 100; number += 2 )
15         sum += number;          // add number to sum
16
17     cout << "Sum is " << sum << endl; // output sum
18     return 0;                  // successful termination
19
20 } // end function main

```

Sum is 2550

Examples Using the for Structure

□ Program to calculate compound interest

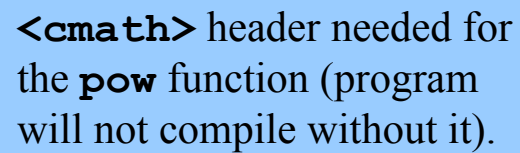
□ *A person invests Rs1000.00 in a savings account yielding 5 percent interest. Assuming that all interest is left on deposit in the account, calculate and print the amount of money in the account at the end of each year for 10 years. Use the following formula for determining these amounts:*

$$a = p(1+r)^n$$

□ *p* is the original amount invested (i.e., the principal),
r is the annual interest rate,
n is the number of years and
a is the amount on deposit at the end of the *n*th year

```
1 // Fig. 2.21: fig02_21.cpp
2 // Calculating compound interest.
3 #include <iostream>
4
5 using std::cout;
6 using std::endl;
7 using std::ios;
8
9
10 #include <iomanip>
11
12 using std::setw;
13 using std::setprecision;
14
15 #include <cmath> // enables program to use function pow
16
17 // function main begins program execution
18 int main()
19 {
20     double amount;           // amount on deposit
21     double principal = 1000.0; // starting principal
22     double rate = .05;       // interest rate
23
```

<cmath> header needed for the **pow** function (program will not compile without it).



```

24 // output table column heads
25 cout << "Year" << setw( 21 ) << "Amount on deposit" << endl;
26
27 // set floating-point number format
28 cout << setprecision( 2 );
29
30 // calculate amount on deposit for each of ten years
31 for ( int year = 1; year <= 10; year++ ) {
32
33     // calculate new amount for specified year
34     amount = principal * pow( 1.0 + rate, year );
35
36     // output one table row
37     cout << setw( 4 ) << year
38         << setw( 21 ) << amount << endl;
39
40 } // end for
41
42 return 0;    // indicate successful termination
43
44 } // end function main

```

Sets the field width to 21 characters, right-justified.

$\text{pow}(x, y)$ = x raised to the yth power.

Year	Amount on deposit
1	1050.00
2	1102.50
3	1157.63
4	1215.51
5	1276.28
6	1340.10
7	1407.10
8	1477.46
9	1551.33
10	1628.89

Numbers are right-justified
due to setw statements (at
positions 4 and 21).